Dit svar IKKE AFLEVERET

Afleveringsopgave

Lab: TensorFlow.js Training and Testing

In this lab we will train and test our linear regression model.
Carry off where you left off at the end of the last lab. Alternatively, there is a reference copy of the code attached as a resource for this lesson which you may use.
Refer to the TensorFlow.js API docs as you complete each step.

Part 1 - Training

1. Add a new function:

```
function trainModel(model, trainingFeatureTensor, trainingLabelTensor) {
  model.fit(trainingFeatureTensor, trainingLabelTensor, {
    epochs: 20,
    shuffle: true,
  });
}
```

2. Call it with `trainModel(model, trainingFeatures, trainingLabels)`
3. Add a callback to print the loss at the end of each epoch.
4. Run in the browser and observe loss being minimized.
5. Remove that callback, and add callbacks to visualise with tfjs-vis:

```
const { onBatchEnd, onEpochEnd } = tfvis.show.fitCallbacks(
  { name: 'Training Performance' },
  ['loss']);

callbacks: {
  onBatchEnd,
  onEpochEnd,
  onEpochBegin: function () {
    tfvis.show.layer({ name: `Layer 1` }, model.getLayer(undefined, 0));
  }
},
```

Extra challenge

Try changing the some of the following options:

- batchSize
- epochs

- validationSplit
- callbacks (disable onBatchEnd)

  Also change the SGD learning rate or try other optimizers (see `createModel()` from previous section)

  Observe the training process.

  Part 2 - Testing

1. Add a new function:

```
function testModel(model, testingFeatureTensor, testingLabelTensor) {
  const lossTensor = model.evaluate(testingFeatureTensor, testingLabelTensor);
  console.log(lossTensor);
  const loss = lossTensor.dataSync();
  console.log(`Testing Loss: ${loss}`);
}
```

2. Call it with `trainModel(model, testingFeatures, testingLabels)`
3. Try it in the browser and verify that the loss is reasonably close to the loss for the training set.